

UNIVERSITY OF CAPE COAST.

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

COURSE CODE: INF 311

COURSE TITLE: DATA STRUCTURES

REGISTRATION NUMBER: PS/ITC/24/0398

LECTURER NAME: MISS LINDA OTOO

Describe real-world applications that require arrays as a data structure, explain why arrays are necessary, and choose one application to develop.

Real-World Applications That Require Arrays

What is an array?

An array is an **ordered, indexed, contiguous** collection of elements: one variable holding many values of the same type.

Consider storing 200 contacts as 200 separate variables (c1 ... c200). You could not list, search, or sort them without rewriting every statement. One array plus a loop replaces all of that code.

Why arrays are necessary

- **Fast access by position.** arr[i] goes straight to item i with no searching. This suits data where position matters, such as ranks, pixels or hours of the day.
- **One loop handles everything.** A single array and a loop work the same for 8 items or 80,000, instead of 200 separate variables.
- **Contiguous memory.** Elements sit side by side, so scans are cache-friendly and pointer arithmetic works. This is why images, audio buffers and sensor logs are stored as arrays.
- **Simple to manage.** An array has a fixed size and a counter n for how many slots are used. Adding at the end is fast; inserting or removing in the middle means shifting items, which is slower.

Six real-world applications (C++)

	Application	How the array is used
1	Phone contacts (app I chose to build)	Add and remove with shifts, substring search, sort A-Z.
2	Classroom gradebook	Average, max/min, sort, filter pass/fail.
3	Digital images	Filters reach neighboring pixels by index arithmetic.
4	Music playlist	Order is the feature; shuffle is a swap of indices.
5	Weather / sensor log	Index math gives "24 hours ago" directly.
6	Leaderboard / tile map	int board[100] kept sorted descending; int map[20][30] for collision lookup.

Chosen application: phone contacts manager

Why this one. A contact book needs everything arrays are good at: an ordered list, instant access by position, scanning every element for a search, sorting, and the cost of shifting elements when one is inserted or removed. It makes the trade-offs of arrays visible.

Data design. Each entry is a struct, and the book is an array of structs with a separate counter for how many slots are in use:

```
struct Contact {
```

```
    char name[30];
    char phone[16];
};

const int CAPACITY = 20;
Contact contacts[CAPACITY];   // fixed-size array
int n = 0;                    // number of used slots (logical size)
```

Operations and cost.

Operation	How it works on the array	Time
addContact	Write at contacts[n], then n++. Fails with an overflow message when n == 8.	O(1)
removeAt(i)	Shift every element after i one slot left, then n--.	O(n)
linearSearch	Scan from index 0 to n-1, counting comparisons, stop on first match.	O(n)
sortByName	Order entries A-Z by name.	O(n log n)
update(i)	Overwrite contacts[i] directly.	O(1)

Core logic.

```
bool addContact(const Contact& c) {
    if (n == CAPACITY) return false;      // overflow: array cannot grow
    contacts[n++] = c;                    // O(1) append
    return true;
}

bool removeAt(int i) {
    if (i < 0 || i >= n) return false;    // invalid index
    for (int k = i; k < n - 1; ++k)       // shift left: O(n)
        contacts[k] = contacts[k + 1];
    --n;
    return true;
}

int linearSearch(const char* key, int& comparisons) {
    comparisons = 0;
    for (int i = 0; i < n; ++i) {         // scan: O(n)
        ++comparisons;
        if (std::strcmp(contacts[i].name, key) == 0) return i;
    }
    return -1;                            // not found
}

void sortByName() {
    std::sort(contacts, contacts + n,     // O(n log n)
        [](const Contact& a, const Contact& b) {
            return std::strcmp(a.name, b.name) < 0;
        });
}
```

The application. The accompanying app is a phone-frame contact manager whose logic mirrors contacts.cpp. The on-screen list shows the live array indices [0..n-1]. An inspector panel shows n/capacity, the number of search comparisons, and whether the array is currently sorted. Every action prints the C++ function it corresponds to, so the user can see an interface action and the array operation behind it side by side.

Conclusion

Arrays are necessary wherever data has a meaningful order, must be reached by position in constant time, and must be processed in bulk with loops: gradebooks, images, playlists, sensor logs, game boards and contact lists. The contacts app shows both sides of the trade-off. Index access and appending are O(1), but arrays do not auto-grow, so inserting or removing an element means shifting its neighbors at O(n) cost. That cost is exactly why vectors and queues exist, and understanding it on a raw array is what makes those structures make sense.